normative_language_cn

规范性语言：如何组织高度复杂系统的规范性描述

状态： 提案草稿，v0.1 **读者：** 从事复杂系统（硬件、协议、大型软件）AI 辅助（agentic）设计的工程师、架构师与研究者。

> **译注：** 本文档是 `normative_language.md` 的中文版本，两者必须保持内容一致。修改任一版本时，须在同一次提交中同步更新另一版本。规范性关键词（MUST/SHOULD/MAY 等）、条款 ID、元数据字段、工具命令与代码示例保留英文原样。

---

1. 问题

1.1 核心问题

> **我们如何组织一个高度复杂系统的规范性描述（normative description）？**

所谓*规范性描述*，指的是*支配*一项设计的陈述总体：系统**应当（shall）**做什么、**不得（must not）**做什么、**可以（may）**做什么，以及在什么条件下。它是判定一个实现正确与否的权威依据。它区别于实现本身、区别于测试集（测试只是对它的采样）、也区别于教程和评注（它们只是对它的解释）。

这个问题由来已久——标准组织与之搏斗了一个世纪——但随着 agentic AI 编程的到来，它变得空前尖锐。一个负责设计或实现系统的 AI agent 需要一份关于设计目标的规范性描述。而今天，尚不存在一种既严谨又切实可行的方法来提供这样的描述。

1.2 人类今天的做法

人类社会把规范性知识编码在*规范文档*中：协议规范（IEEE 802.3、PCIe、ARM AMBA CHI）、API 规范、硬件模块架构规范、标准与数据手册。这些文档具有一些共同特征：

1. **为人类阅读而写。** 它们依赖散文、表格、时序图、状态机插图和波形图。对它们的解读需要人类的判断力和领域背景。
2. **满载历史补丁。** 像 PCIe 或 IEEE 802.3 这样的成熟规范，承载着数十年修订史中积累下来的勘误、增补、可选特性和废弃条款。文档结构反映的更多是其编辑历史，而非系统的逻辑结构。
3. **天然只是局部描述。** 每份文档只描述系统的一个切片。以太网交换机的设计者必须把散布在众多 IEEE 802 文档（802.3 PHY/MAC、802.1Q 桥接/VLAN、802.1AX 链路聚合……）以及厂商数据手册中的知识聚合起来，而*交换机本身*的描述相对于这座参考资料大山而言微不足道。**这种聚合发生在人类读者的头脑中，从未被写下来。**
4. **交叉引用是非形式化的。** "见 7.3.2 节"和"如 [802.1Q] 所定义"对细心的人类读者是可解析的，但脆弱、无法验证，而且经常过时。

1.3 今天的 AI 辅助设计如何运作——以及什么东西丢失了

当前人类与 AI 协作设计的实践是一个迭代循环：

1. 人类提供一份*不完整的*规范性描述（一个初始 prompt，或许再加一份设计文档）。
2. Agent 产出一个局部设计，或针对当前设计的测试报告。
3. 人类观察输出，提供*追加的* prompt：澄清、纠正、新需求、方向调整。
4. 如此往复，可能历经数百个周期，直到 agent 收敛出一个完整的项目。

请注意，最终系统的*真正*规范性描述由什么构成：

> **背景参考文档 + 初始设计文档 + 整个 prompt 与纠正序列**

这个聚合体才是支配了设计的真正"圣经"。然而：

- 它**不以某种集中的形式存在**。它散落在聊天记录、文档和人类记忆之中。
- 它**无法被规范地重放**。Prompt 序列是顺序相关的，包含依赖上下文隐式化解的矛盾，并且与 agent 的输出纠缠在一起。
- 它**通常会丢失**。聊天记录被丢弃，没有版本管理，没有维护。没有修订控制，无法追问"哪条需求导致了这个设计决定？"或"我们在第 40 个周期里对 X 做了什么决定？"
- 后来的 prompt **静默地覆盖**先前的 prompt，而这种取代没有任何记录。

结果是：系统存在了，但它的规范性描述不存在。维护、验证和派生设计都因此受害。此后的每一次修改都要重新做一遍考古。

1.4 我们需要什么

我们必须定义一种**组织规范性描述的规范格式与方法**，它要能：

- **增量地构建起来**——我们不可能一次写成；
- 在**整个项目周期中持续修订**，并保留完整的**演化历史**；
- 让人类（和 agent）在项目结束时收敛出一份**比开始时更完整的描述**；
- 支持把**面向人类的参考文档转换**为该格式；
- 以**层级结构（树）组织以便维护，并叠加简便、一致的交叉引用**（树之上的图）；
- 做到**人和机器同时可读**；
- 支撑由人类成员与 AI agent 成员协作进行的、**演化式的、多周期迭代的** agentic 设计 / 验证 / 物理实现循环。

---

2. 设计哲学：中间道路

2.1 两种失败模式

描述系统行为的严格程度存在一个谱系，而两个极端在实践中都会失败：

失败模式 A——纯散文（现状）。 为人类书写的自然语言文档：含混、不可合并、不可查询、不一致，而且——如上文所论——在 AI 时代的工作流中甚至没有被收拢在一处。

失败模式 B——完全形式化。 另一个极端：用形式化规范语言描述整个系统——ARM 的 ASL（Architecture Specification Language，"规范即代码"）、TLA+，或 Lean、Coq 这样的证明助手。这强加了一种*令人望而却步*的严格性：

- 编写并维护形式化规范的工作量可能**超过设计本身的工作量**。（ARM 能为 ISA 负担得起 ASL，是因为 ISA 的成本可以摊销到数十亿颗芯片上；一个设计单台交换机的项目团队做不到。）
- 形式语言擅长刻画*功能性*行为，却难以表达性能意图、物理约束、成本权衡以及"应当合理"这类工程判断——而这些恰恰构成真实规范的很大部分。
- 形式化把决策前置。项目早期，规范*必然是含混的*；形式语言没有办法做到"有用地含混"。
- 能读懂规范的人群会急剧萎缩。

我们明确拒绝这两个极端。

2.2 中间道路：结构化自然语言，代码作为一等公民

我们的立场：

1. **自然语言仍是主要载体。** 它是唯一有足够表达力去承载早期含混性、设计依据（rationale）和非功能性意图的媒介——而且，人类历史上第一次，机器也能读懂它了。LLM 改变了这道算术题：结构化的自然语言不再是"机器不可读"的。
2. **结构施加在*组织*层面，而非*句子*层面。** 我们不约束一句话怎么写（除了规范性关键词约定，见 §4.4）；我们严格约束的是陈述如何被*标识*、*分类*、*连接和版本化*。
3. **凡是代码比散文更便宜的地方，都鼓励用代码。** 一个 20 行的 Python 仲裁算法参考模型，比一页散文更精确，*而且*写起来更快。报文格式声明、状态表、伪代码片段——这些是嵌在散文海洋中的"形式化岛屿"。我们汲取"规范即代码"的智慧，而不承担其全盘化的代价。
4. **承认不完美，并为之做工程设计。** 人类思维并非无瑕；一份彻底完整、一致、无缺陷的规范性描述是不可企及的。方法论的目标不是完美，而是**单调的质量改进**：每个周期都应让规范比之前更完整、更一致、经过更充分的交叉校验。因此格式必须支持对已知缺口、开放问题和未决矛盾的显式标记——一份能坦诚说出"这部分尚未决定"的规范，比一份说不出口的规范更可信。

2.3 规范即一个收敛的程序序列

有一个值得上升为原则的关键想法。考虑把完整规范性规范的开发过程当作**编写一系列程序**：

- $\mathbf{P_0}$ 是含混的、高层的：大部分是自然语言"注释"、没有函数体的函数签名、类型为 `TODO` 的空洞。它只在读者（或 agent）的头脑中"运行"。
- $\mathbf{P_1}$、$\mathbf{P_2}$**……** 各自补充进一步的细节：一些空洞获得了伪代码，一些伪代码变成了可执行的参考模型，一些散文变成表格再变成数据结构。
- **P_n**，序列的极限，是一个实现了完整系统的程序——*产品本身*。

这个想法有深厚的渊源：它正是**逐步求精**（stepwise refinement，Wirth，1971）和**精化演算**（refinement calculus，Back、Morgan）——一个含有规范语句的抽象"程序"被逐步精化为可执行代码，每一步都保持正确性。文学式编程（literate programming，Knuth）贡献了互补的洞见：散文与代码应当共处于同一份文档中，按人类理解的顺序编排。

我们把它*作为一种思维模型和组织手段*来采纳，并做两个关键的放松以保持其可行性：

1. **精化步骤不做形式化验证。** 我们不证明 P_{k+1} 精化了 P_k；我们只*记录*精化关系（条款 X 细化了条款 Y），把违规交给评审、测试和 agent 交叉校验去发现。是簿记上的严格，不是证明上的严格。
2. **序列永远不会完全坍缩为产品。** 实践中，规范与实现始终是处于不同抽象层次的两个不同工件；规范中靠后的那些"程序"是*参考模型*和*可执行的验收检查*，而不是交付的 RTL 或生产代码。这个序列的价值在于**每一层都持续存在并相互链接**——含混的 P_0 不会在 P_3 出现后被丢弃；它仍是可读的摘要，而层与层之间的链接就是可追溯性。

具体而言，这意味着格式要支持**精化层（refinement layers）**（§4.6）：同一行为在多个抽象层次上被描述，层间显式链接，从一句话的意图一直到可执行模型。

---

3. 先例，以及我们从每一项中各取什么

一份简要综述；每一行列出我们采纳什么、拒绝什么。

来源	是什么	我们采纳	我们拒绝
IETF RFC / RFC 2119	规范性关键词约定（MUST/SHOULD/MAY）、编号章节、勘误流程	关键词纪律；"规范性 vs. 资料性"文本的文化	单体式扁平文档；靠勘误打补丁
IEEE / PCIe / ARM 规范	成熟的多卷本规范	符合性条款的概念；PICS（协议实现符合性声明）作为可机器检查的声明清单	由修订史驱动的文档结构；只有人类能看的图表
ARM ASL、TLA+、Lean	完全形式化的规范语言	可执行的岛屿；"精确性*可以*是代码"这一理念	把全盘形式化作为准入门槛
EARS 记法（Easy Approach to Requirements Syntax）	受约束的自然语言需求模板（"When ⟨trigger⟩, the ⟨system⟩ shall ⟨response⟩"）	可选的句式模板，仅作为 *lint 建议*，绝不作为门禁	对所有陈述强制套用模板

来源	是什么	我们采纳	我们拒绝
Doorstop / ReqIF / DOORS	需求管理：条目以文件形式存于版本控制、文档树、链接校验	条目级身份标识；由工具检查链接；以 VCS 作为历史机制	一条目一文件的粒度（对散文为主的规范过于碎片化）；以 YAML 作为撰写界面
规范驱动开发工具（GitHub Spec Kit、AWS Kiro、Tessl，2025–26）	驱动编码 agent 的 Markdown 规范脚手架（spec/plan/tasks）	以 Markdown 为基底；先规范后代码的工作流；不可妥协项的"constitution"	只在一次变更请求生命周期内存活的按特性、按分支的规范——我们需要*锚定于系统整个生命周期的规范*
文学式编程	散文与代码交织，按人类阅读顺序编排	代码块作为一等的规范性内容	tangle/weave 工具链的复杂性
逐步求精 / 精化演算	从抽象到具体的程序序列	分层精化的思维模型（§2.3）	每一步的证明义务
ADR（架构决策记录）	记录决策与上下文的小型不可变记录	用决策记录模式捕捉*为什么*，包括从 prompt 日志蒸馏出的决策	—
Git + Markdown 生态	无处不在的纯文本版本管理	整个持久化与历史层	另行发明一个数据库

所有先例共同的缺口：没有任何一个方案同时做到 (a) 把*整个系统*的规范性描述当作一等的、以系统生命周期为尺度的、受版本管理的工件，(b) 停留在切实可行的严格度之内，(c) 天然可被人类和 AI agent 共同消费，以及 (d) 提供*摄取*既有的、面向人类的海量参考文档的路径。这正是本提案的目标。

4. 提案：NDF——一种规范性描述格式（Normative Description Format）

我们提出 **NDF（Normative Description Format）**：一套叠加在 Markdown + Git 之上的约定，外加一个小型工具链。它不需要任何新的文件格式或服务器；一个纯文本编辑器加 `git` 就足以撰写——这恰恰是关键所在。

4.1 整体形态概览

```
spec/                          # 规范根目录（"圣经"）
  ndf.yaml                     # 清单：ID 前缀、层名、lint 配置
  00-charter/                  # 是什么与为什么：范围、目标、非目标、术语表
  10-architecture/             # 系统分解、框图（文本 + 图片）
  20-behavior/                 # 主体：按子系统组织的规范性行为条款
     20-ingress/
        pipeline.md
        parsing.md
     21-forwarding/
        ...
  30-interfaces/               # 内外部接口契约
  40-constraints/              # 性能、资源、物理、成本约束
  50-verification/             # 验收准则、符合性清单（类 PICS）
  models/                      # 可执行参考模型（代码），由条款链接
  refs/                        # 摄取的外部参考资料（§6）
     ieee-802.3/
     ieee-802.1q/
  decisions/                   # 决策记录（ADR 风格），含从 prompt 日志蒸馏的内容
  open/                        # 开放问题与已知矛盾，作为条目追踪
```

三种结构共存：

1. **树**——目录/标题层级。这是*归属与维护*结构：每个条款有且仅有一个家，树是人类导航的方式，也是编辑责任划分的方式。
2. **图**——条款之间带类型的交叉引用（`refines`、`depends-on`、`conflicts-with`、`verifies`、`derived-from`、`couples-with`，外加普通提及）。这是*语义*结构；它是非树的，可以自由穿越层级。
3. **历史**——Git 提交，加上显式的条款级修订标记与决策记录。这是*演化*结构。

一句话概括这套纪律：**散文活在树里，语义活在图里，时间活在 git 里——而稳定的条款 ID 是把三者铆在一起的铆钉。**

4.2 基本单元：条款（clause）

原子性的规范单元是**条款**：Markdown 文件中一个以标题划界的块，携带一个**稳定的、全局唯一的 ID**。其解剖结构：

```
## Frame admission on ingress {#FWD-ADM-001}
<!-- ndf: kind=req level=must layer=L1 status=stable since=0.3
-->

When a frame arrives on an ingress port, the switch MUST admit
it to the
forwarding pipeline only if all of the following hold:

1. the frame passes FCS validation ([[ING-FCS-002]]);
2. the frame length is within [minFrameSize, maxFrameSize]
   as configured per port ([[CFG-PORT-007]]);
3. the ingress port is in `forwarding` state per the
applicable
   spanning-tree instance ([[refs/ieee-802.1q#8.6.1 | 802.1Q
§8.6.1]]).

Frames failing any condition MUST be discarded and the per-
port
`admissionDrops` counter ([[TEL-CNT-014]]) MUST be
incremented.

> rationale: FCS-invalid frames must not consume pipeline
credits;
> see [[decisions/D-0042]] for the credit-accounting
discussion.
```

规则：

- **ID**（`{#FWD-ADM-001}`）：一次分配，永不复用，永不重编号，跨文件移动后依然存续。ID 按区域加前缀（在 `ndf.yaml` 中声明），由工具签发（`ndf new-id FWD-ADM`），因此 agent 和人类不会发生冲突。
- **元数据注释**：机器可读，单行，刻意保持极简：
  - `kind` —— `req`（需求）、`def`（定义）、`arch`（架构陈述）、`constraint`（约束）、`option`（推迟到探索阶段确定的设计空间参数，见 §4.8）、`verif`（验收准则）、`info`（显式的非规范性内容）。
  - `level` —— `must` / `should` / `may` / `tbd`（RFC 2119 风格；`tbd` 是合法且诚实的取值）。
  - `layer` —— 精化层，见 §4.6。
  - `status` —— `draft` / `stable` / `deprecated` / `superseded-by=ID`。
  - `since` —— 该条款当前实质内容进入规范时的版本号。

- **交叉引用**使用 `[[ID]]`（可选加 `| 显示文本`）。工具负责解析，遇到悬空引用即令构建失败，并生成反向链接索引。普通引用之外的带类型的边写在元数据中：`<!-- ndf: refines=FWD-ADM-000 verifies=... -->`。
- **规范性关键词**（MUST/SHOULD/MAY/MUST NOT）按 RFC 2119 在 `req` 条款内使用；linter 会标记不含任何关键词的 `req` 条款，也会标记出现在 `info` 条款中的 MUST/SHALL。

条款块之外的一切——引言散文、图、示例——默认为资料性（informative）。**规范性与资料性的边界由此在语法上被显式化**，而这正是传统规范中最大的一处含混。

4.3 为什么用标题锚定的条款，而不是一条目一文件

Doorstop 式的"一条需求一个 YAML 文件"最大化了机器可处理性，却摧毁了可读性和撰写体验——没有人会以一句话一个文件的方式写出好散文，而在这里散文质量是承重的。标题锚定的条款让文件作为文档保持可读（人或 agent 可以从头到尾读完 `parsing.md`），同时 ID 纪律让条目作为数据保持可寻址。工具随时可以把树*炸开*为逐条目的记录（JSON）以供查询；而撰写形态始终保持人类的形状。

4.4 自然语言，温和地约束

我们不强制句式模板。我们通过 linter 提供*建议性的*精确化压力：

- 当某个 `req` 条款被标记为含混时，提供 EARS 风格的句式建议（"考虑改写为：*When ⟨trigger⟩, ⟨system⟩ shall ⟨response⟩ within ⟨bound⟩*"）。
- 项目**术语表**（`00-charter/glossary.md`）由 `def` 条款构成；linter 标记未定义的专业术语，以及不一致的同义词混用（"packet" vs "frame"）。
- 对 `must` 级条款设置已知歧义词黑名单（*appropriately*、*as needed*、*etc.*、*handle*、*support*（不带宾语时））。

建议性而非阻断性：人类（或经人类签核的 agent）总是可以保留被标记的句子。施加质量压力，但不筑起严格性的高墙。

4.5 代码作为一等的规范性公民

条款正文可以包含被标记为规范性的代码块：

```
## Weighted round-robin egress scheduling {#SCH-WRR-003}
<!-- ndf: kind=req level=must layer=L2 model=models/sch_wrr.py
-->
```

```
        Egress scheduling among queues of equal priority MUST follow
        weighted round-robin with per-queue weights `w[i]`, deficit-
counter
        variant, as specified by the reference model:

        ```python ndf:normative
        # Simplified normative model — authoritative for ordering
semantics,
        # not for performance. models/sch_wrr.py is the executable
version.
        def select_next(queues, deficit, quantum):
            for i in rotate(range(len(queues)), state.last):
                if queues[i].empty(): continue
                deficit[i] += quantum[i]
                if queues[i].head().size <= deficit[i]:
                    deficit[i] -= queues[i].head().size
                    return i
            return None
        ```
```

约定：

- ```python ndf:normative 把代码块标记为规范性的：它*定义*行为，而不只是举例说明。未标记的代码块是示例（资料性）。
- `model=models/sch_wrr.py` 把条款链接到一个**可执行参考模型**，模型放在 `models/` 下保持可运行，并带有自己的测试。条款内嵌的片段是可读的摘要；被链接的模型才是权威的可执行版本。构建会运行模型测试，因此模型坏了规范构建就会失败——这是形式化一致性检查的务实替代品。
- 数据形态的内容（报文格式、寄存器映射、状态表、配置 schema）SHOULD 采用代码/数据而非散文表格：结构体定义、JSON Schema、由工具渲染成表格的 CSV。源头机器可读，出版时美观。

这正是"规范即代码"以近乎零边际成本兑现价值的地方：只在代码是最便宜的精确记法的地方写代码，其余地方一概不写。

4.6 精化层：程序序列的具体化

每个条款携带一个 `layer` 标签。建议的默认阶梯（项目可通过 `ndf.yaml` 重命名）：

- **L0——意图。** 一到几句话。"交换机在 N 个端口之间以线速转发以太网帧，透明地学习地址。" 不涉及机制。
- **L1——行为契约。** 外部可观察的行为，精确但不涉及机制。大多数 `req` 条款位于此层。
- **L2——机制。** 选定的算法、数据结构、内部分解。WRR 调度、基于哈希的 MAC 表、流水线级。

- **L3——可执行模型。** `models/` 下的参考实现、golden vector、验收测试规范。

让它成为一个*序列*而不是一堆杂物的规则：

- 低层编号的条款由高层编号的条款通过显式的 `refines=` 边来精化。`ndf trace FWD-ADM-000` 打印精化子树——该行为的"程序序列"。
- **所有层都保持存活。** L2 出现后 L0 绝不删除；它仍是人类和 agent 最先阅读的摘要。当一处 L2 修改与其 L1 父条款矛盾时，linter 标记这一*对*条款要求调和——有时该改的是 L2（机制错了），有时该改的是 L1（契约真的变了，而这必须是一个可见的、有意为之的动作）。
- **覆盖率变得可度量。** "哪些 L1 契约还没有 L3 验收准则？"成了一个查询（`ndf coverage`），把"规范够完整了吗？"从一种感觉变成一份报告。完整性永远不会是全然的（§2.2 第 4 点已承认），但其*边界变得可见*。
- 项目早期，整个规范可能都是 L0/L1，到处点缀着 `level=tbd`。这是一个*合法的、可构建的状态*——含混性是可表示的，而这恰恰是形式化方法给不了的。

4.7 显式地处理不完美

因为我们接受规范永远不会完整或完全一致，不完整性是*结构化的*，而不是被掩盖的：

- **开放问题**是 `open/` 下的条目，各有 ID（`Q-017`），其阻塞的条款（`blocks=FWD-ADM-001`）和一个决议字段。`ndf status` 列出它们。解决一个开放问题，会在同一次提交中产生一条决策记录和一处条款修改。
- **已知矛盾**：当发现两个条款相互矛盾时，发现者（通常是 agent）添加一条 `conflicts-with` 边和一个 `Q-` 条目，而不是悄悄选一个赢家。构建会警告但不会失败——真实项目会带着已知矛盾生活数周，假装不是这样只会把矛盾逼入地下。
- **TBD 空洞**：`level=tbd` 的条款和行内的 `⟨TBD: max latency bound⟩` 标记会被计数并报告。发布门禁可以要求已交付区域的 `must` 条款中 TBD 数为零——*项目*选择自己的门禁；*格式*只负责让空洞可计数。
- **设计选项（design option）**（§4.8）是一类独立而更丰富的开放条目：其*取值或机制选择*被推迟到设计空间探索阶段确定，但它携带一个默认值和一个允许范围，因此在其保持开放期间不会阻塞任何工作。它们与 TBD、开放问题一并被计数。

4.8 设计选项：推迟到设计空间探索的参数

硬件与系统建模中反复出现的一种情形：某个量在模型开发阶段确实尚未确定，但它并不是一处*意图上的缺口*（§4.7）——意图是清楚的（"存在一个重排序缓冲区，它有一个深度"）。开放的是一个**取值，或若干机制之间的选择**，留待后续由*设*

计空间探索（design-space exploration，DSE）来敲定：仿真、扫描、成本/性能/面积的权衡研究。例子：发射队列（issue queue）与重排序缓冲区（reorder buffer）的深度、每 bank 带宽、每个处理单元（PE）支持的同时线程数、仲裁策略（轮询 vs. 按年龄 vs. 按优先级）。

关键的不对称性在于：**这样一个量，对 RTL 实现者是常量，对模型开发者却是变量。** RTL 冻结为单一取值；模型则扫描一个范围，让仿真结果来选。规范格式必须让两类读者看到*同*一条款，各取所需——既有可据以构建的取值，*也有*仍待探索的空间。

我们把它们提升为一等公民，即 `option` 条款类型。一个 `option` 条款命名一个设计空间参数，并记录：

- 一个**默认值（default）**——在 DSE 另有结论之前，性能模型和任何早期 RTL 应假定的取值（`default=`）；
- 一个**探索范围（exploration range）**——该参数在 DSE 期间允许取的离散集合或区间（`explore=`）；
- 可选的**耦合关系（coupling）**——`couples-with=` 边，指向那些取值*并不独立*的其他选项，并在散文中陈述其因果/权衡关系。参数之间常常相互约束（例如线程间共享的一个固定总资源预算），逐一维度地探索会错过真正的最优点。

```
## Per-thread reorder-buffer depth {#PRM-ROB-001}
<!-- ndf: kind=option level=tbd layer=L2 status=draft
since=0.4 -->
<!-- ndf: default=64 explore=32,48,64,96,128 unit=entries -->
<!-- ndf: couples-with=PRM-SMT-001 depends-on=PRM-IQ-001 -->

The reorder-buffer depth allocated **per hardware thread** is
a
design-space parameter — **TBD, to be determined by design-
space
exploration.** It is a fixed constant in RTL and a swept
variable in
the performance model.

- **default:** 64 entries — assume this until DSE closes the
option.
- **exploration range:** {32, 48, 64, 96, 128} entries.
- **coupling:** trades off against the SMT thread count
  ([[PRM-SMT-001]]) under a fixed total ROB budget — the more
threads a
  PE supports concurrently, the smaller each thread's ROB (and
issue
  queue, [[PRM-IQ-001]]) can be. DSE MUST explore the pair
jointly,
  not each axis in isolation.
```

```
        > rationale: fixing this before the SMT width is known would prejudge
        > the trade-off; see [[decisions/D-0101]] when the sweep resolves it.
```

约定与生命周期：

- `option` 条款在开放期间通常是 `level=tbd`。这**不是缺陷**——与一个光秃秃的 ⟨TBD⟩ 空洞不同，它携带一个可用的默认值，因此模型和 RTL 永远不会被阻塞：它们基于默认值继续推进，并记录下这么做了。
- **行内选项。** 当某个参数出现在另一个条款内部、不值得单列一条时，使用行内标记 ⟨DSE: <name> default=X range=… → PRM-ID⟩，即 ⟨TBD: …⟩ 的设计空间对应物。二者都被普查（§4.7）计数。
- **耦合关系在散文中显式且有方向。** `couples-with` 边是对称的簿记；依赖的*本质*——它往哪个方向推、在什么固定预算之下——写在正文里，好让 DSE 和评审者对联合空间进行推理，而不是一次只看一个变量。
- **关闭一个选项是一次有记录的事件，而非静默覆盖。** 当 DSE 选定一个取值时，该决议在同一次提交中落地为一条决策记录加一处条款修改（§5.2）：`default=` 变为选定的取值，`explore=` 作为历史保留（或收窄），`level` 从 `tbd` → `must`（它现在是一份固定的契约），`status` 从 `draft` → `stable`。此后，RTL 常量与设定它的那次探索相互链接。
- **选项是可查询的。** `ndf options`（§8）列出每个开放参数及其默认值、范围和耦合图——生成的 DSE 工作清单。`ndf coverage` 把开放选项与 TBD、开放问题一并计数，于是设计空间的边界与不完整性的边界一样可见。

这守住了中间道路（§2）：参数的存在与意图现在就被钉住，以人类可读、模型可消费的散文写下；而那唯一诚实开放的自由度——它的取值——被表示为一个有界、有默认、带耦合的变量，而不是一个虚假的常量或一处未受追踪的空洞。

---

5. 演化与修订追踪

规范在整个项目中持续被修订；历史不是事后补丁，而是格式设计的驱动因素。

5.1 以 Git 为基底，其上叠加规范感知的语义

Git 中的纯文本 Markdown 已经给了我们：完整历史、支持并发编辑的分支/合并（当多个 agent 并行工作时至关重要）、blame、作为基线的 tag，以及作为规范变更人工审批关口的 PR 评审。我们在其上添加规范感知的语义：

- **条款级历史。** 由于 ID 稳定且锚定于标题，`ndf log FWD-ADM-001` 能跨文件移动和重命名重建一*个条款*的历史——"这条需求是如何演化的？"得到了一等公

民级的回答，而对文件裸跑 `git log` 给不出这个答案。

- **语义 diff。** `ndf diff v0.3..v0.5` 报告：新增 / 实质修改 / 废弃 / 被取代的条款；增删的边；TBD 计数变化；覆盖率变化。这就是规范基线之间的变更日志——生成的，不是手写的。
- **基线。** 打了 tag 的版本（`spec-v0.5`）是*设计*所引用的单位：一次实现、一轮验证战役或一次流片，都记录其针对的规范基线。这把从工件回溯到确切支配文本的闭环补上了。
- **取代，而非删除。** 被替换的条款获得 `status=superseded-by=NEW-ID` 并留在树中（出版时可选移入归档章节）。推理的痕迹得以存续。

5.2 决策记录：捕捉*为什么*，包括 prompt 流

Prompt 日志问题（§1.3）的解法不是归档原始聊天记录（不可重放、噪声大），而是**蒸馏为决策记录**。`decisions/D-0042.md`：

```
# D-0042: Credit accounting excludes FCS-invalid frames {#D-0042}
<!-- ndf: kind=decision date=2026-07-14 affects=FWD-ADM-001,ING-FCS-002 -->

**Context.** During cycle 12 of pipeline design, agent testing showed
credit leakage when malformed frames consumed pipeline credits before
FCS check completed (test report: verif/runs/r-0231).

**Decision.** FCS validation moves ahead of credit acquisition;
FCS-invalid frames never consume credits.

**Alternatives rejected.** Post-hoc credit refund (races with
back-pressure, rejected); oversized credit pool (hides the bug).

**Source.** Human–agent session 2026-07-14; superseding instruction
in the same session overrides the cycle-9 guidance to "check FCS late."
```

工作流规则：**任何改变设计意图的、由人类发给 agent 的指令，必须在同一次提交中落地为一处条款修改或一条决策记录（通常两者都有）。** 决策记录由 agent 自己起草——把会话中的规范性内容转化为一次待审的规范提交，正是 agent 擅长的那类总结工作，人类负责审查 diff。转瞬即逝的 prompt 流就这样被持续地*编译*进持久的规范。任何规范性的东西都不允许只活在聊天记录里。

5.3 合并纪律

多个人类/agent 的并发编辑不可避免。缓解措施按重要性排序：(1) 树为每个区域指定一个归属文件，并行工作自然落在互不相交的文件上；(2) 工具签发的 ID 防止标识符冲突；(3) `ndf check` 在每次合并的 CI 中运行，捕获文本合并看不见的悬空引用、重复 ID 和规范性关键词违规；(4) 语义冲突（两个分支分别修改了由 `refines` 边相连的条款）由检查器浮出为评审标记。

---

6. 摄取面向人类的参考文档

回到以太网交换机的场景：设计自身的规范很小，被引用的外部规范却极其庞大。我们需要工具和方法，把为人类读者书写的文档转换为规范性描述的形态——但把 IEEE 802.3 *全部*转换过来既不可行也不可取。

6.1 原则：只导入*所需的投影*，保留指回源头的指针

对每份外部参考资料，我们在 `refs/` 下构建一个**投影（projection）**：本项目实际依赖的外部规范子集，重构为 NDF 条款，每条携带溯源信息：

```
## Frame check sequence computation {#R8023-FCS-001}
<!-- ndf: kind=req level=must origin="IEEE 802.3-2022 §3.2.9"
origin-status=verbatim -->
```

- `origin` 钉住确切的源条款和版本；`origin-status` 取值为 `verbatim`（忠实转述）、`paraphrase`（结构重组，需谨慎对待）或 `interpretation`（我们化解了一处歧义——标记出来供评审，同时也是一条可向上游报告的候选勘误）。
- 投影是*按需驱动的*：agent 在 VLAN 标签处理上遇到问题，触发的是对相关 802.1Q 条款的摄取，而不是整份文档。投影的增长速度恰好等于项目真实依赖边界的推进速度——这正是从前只发生在资深工程师头脑中的那种聚合（§1.2 特征 3）的书面化形态。
- 项目本地条款像引用其他条款一样引用 `refs/` 条款（`[[R8023-FCS-001]]`），使项目的外部依赖面*可枚举*：`ndf deps refs/` 精确列出设计倚赖了哪些标准的哪些部分——当某个标准改版时，这价值千金。

6.2 摄取管线（agent 辅助，人类审计）

1. **抽取。** PDF → 结构化文本（标题、表格、插图作为资产保留）。编码了状态机、格式或参数的表格 → CSV/结构化数据。
2. **分段与分类。** Agent 把文本切分为候选条款，并区分规范性与资料性（在成熟标准中，"shall" 句式使这项工作变得可处理）。

3. **转述。** Agent 把每条保留的条款转述为 NDF 形式，标注 `origin` 并提议 `kind/level`。图表配上散文转述外加原图；状态机转为表格或代码形式。
4. **审计。** 人类（或第二个独立的 agent 运行——廉价的交叉校验是 agent 时代的真正红利）抽样评审，优先处理 `paraphrase/interpretation` 条目。版权说明：投影是为内部工程用途对*技术内容*的转述；关于再分发，团队须自行做出许可上的判断。

这条管线在构造上就是不完美的，而这没有关系：对一个标准所需的那 5% 做出的 90% 忠实、全链接、可查询的投影，胜过一份 100% 忠实、却没有任何工具和 agent 能以条款粒度寻址的 PDF。

---

7. 协作循环：人类和 agent 如何使用这套东西

覆盖设计、验证与物理实现的目标工作流：

1. **引导启动。** 人类撰写 `00-charter` 和一个 L0/L1 骨架——以天计，不以月计。`ndf init` 搭好脚手架；摄取（§6）开始按需拉入参考投影。
2. **设计周期。** 每个 agent 任务被表述为：*基线 `spec-vX` + 一份引用条款 ID 的工单*。Agent 阅读相关子树及其图邻域（条款结构正是让检索变得精确的东西——agent 可以拉取 `FWD-*` 及沿 `refines/depends-on` 边一跳可达的一切，而不是把一份 400 页的 PDF 塞进上下文）。
3. **反馈编译。** 设计产出、测试报告和人类纠正以规范提交的形式回流：条款修改、新的 L2/L3 精化、决策记录、新的 `Q-` 条目（§5.2）。第 k 个周期之后的规范，严格地比周期之前信息更充分。
4. **验证。** `verif` 条款（`50-verification/`）构成符合性清单——PICS 的对应物。每条 `must` 级 L1 条款最终都应被至少一条验收准则以 `verifies` 链接；`ndf coverage` 报告缺口。测试失败引用条款 ID；由失败测试发现的歧义成为一个 `Q-` 条目，而不是一次耸肩。
5. **实现周期**（RTL、物理、软件）在提交信息和设计评审中引用条款 ID，于是反向的问题——"这是哪条需求驱动的？"——变得可以 grep。
6. **出版。** `ndf publish` 把树渲染为 HTML/PDF，带已解析的交叉引用、反向链接索引、追踪矩阵和逐基线的变更日志——对人类友好的"书籍视图"，由规范形态生成，而非规范形态本身。

相对于格式，人类与 agent 的角色是对称的——都读条款、都提议提交；相对于权威，二者是不对称的：人类批准规范性变更（PR 评审），agent 提议并交叉校验。规范是让多 agent、多人类、跨数月的协作保持连贯的共享记忆。

---

8. 工具链（最小可行）

刻意做小；每一件都是直白的工程：

工具	功能
`ndf new-id`	签发无冲突的条款 ID
`ndf check`	Lint：悬空引用、重复 ID、关键词纪律、层间一致性标记、术语漂移；在 CI 中运行
`ndf trace ID`	打印某条款的精化/依赖子树
`ndf log ID`	跨文件移动的条款级历史
`ndf diff A..B`	基线之间的语义变更日志
`ndf coverage`	L1→验证覆盖率；TBD 普查；开放 `option` 普查；`Q-` 状态
`ndf deps`	外部参考依赖面
`ndf options`	列出设计空间参数：默认值、范围、耦合图、开放/关闭状态（§4.8）
`ndf export`	把树炸开为 JSON 记录（供查询、RAG 索引、仪表盘）
`ndf publish`	渲染书籍视图（HTML/PDF），含追踪矩阵
`ndf ingest`	Agent 辅助的参考文档投影（§6.2）
`models/` 运行器	把参考模型及其测试作为规范构建的一部分执行

一切都在纯文件上运作；没有服务器，没有数据库。JSON 导出是通往团队所用的任何 agent 框架或搜索索引的桥梁。

---

9. 诚实的局限与开放问题

1. **一致性靠社会性与经验性手段检查，而非证明。** 两条散文条款可以微妙地相互矛盾，同时通过所有 lint。缓解手段——agent 交叉阅读（如今很廉价）、可执行模型、验证链接——能减少但永远无法消除这一点。这是中间道路（§2.1）被接受的代价。
2. **蒸馏会丢失信息。** 把 prompt 流编译为决策记录是有损的；一些上下文会消亡。我们判断这笔交易是划算的（持久但有损，胜过完整但不可用），但需要处理争议的团队可能希望把原始会话档案作为冷存储保留，由决策记录引用。

3. **纪律的衰减。** 与所有约定一样，如果提交绕过 `ndf check`，或 agent 跳过反馈编译步骤，NDF 就会退化。对策是 CI 强制执行，以及让 agent 工作流*默认*规范优先；真正的对策是文化。
4. **粒度的拿捏。** 一个条款该多大？太细 → 簿记噪声；太粗 → 寻址失去意义。我们提供启发式（每个 `req` 条款一项可测试的义务），但这仍然是一项编辑技艺。
5. **图表密集的内容。** 时序图和波形图抗拒文本化。过渡方案：原图 + 规范性散文转述 + 尽可能提供表格/代码等价物；更好的答案有待工具化（例如规范性的类 WaveJSON 记法）。
6. **精化一致性的缺口。** 我们记录 `refines` 边但不验证它们；L2 变了而 L1 陈旧，只能靠评审或测试发现。未来一种 lint（由 agent 驱动的父子条款语义比对）是可行的，将是一次重大升级。
7. **生态引力。** 格式的价值随工具和习惯而复利增长；孤独的采用者在没有网络效应的情况下大约能获得 60% 的价值（结构、历史、agent 检索精度）。

---

10. 执行计划

阶段 1——格式冻结（第 1–2 周）。 把 `ndf.yaml` schema、条款文法、ID 规则、边类型、层阶梯写成一份简短的规范性文档——*这份文档本身就用 NDF 书写*，第一次吃自己的狗粮。

阶段 2——最小工具链（第 2–6 周）。 基于 Markdown 解析器用 Python 实现 `check`、`new-id`、`export`、`trace`、`publish`；给出 CI 配方。（Doorstop 和 Spec Kit 已演示了所需的每一项技术；这是组装，不是研究。）

阶段 3——真实设计试点（第 4–12 周，与前重叠）。 一个有边界但真实的目标——例如一个 4 端口 L2 以太网交换机模型——以规范优先的方式运行 agentic 设计周期：工单引用条款 ID，反馈编译为规范提交，覆盖率受追踪。试点的度量就是 §11 的问题清单。

阶段 4——摄取工具（第 8–16 周）。 用试点的真实参考资料（802.3/802.1Q 子集）运行 `ndf ingest`；测量每条摄取条款的审计负担。

阶段 5——复盘与修订（第 16 周起）。 试点从起点到终点的规范语义 diff *本身就是证据*：规范性描述收敛了吗？一个仅从 `spec-final` 出发的新团队（或新 agent），能否复现设计的意图？

11. 成功标准

如果试点结束时满足以下条件，方法论即告成功：

1. **圣经存在了。** 一个受版本管理的工件回答"什么支配着这个设计？"——不再需要聊天记录考古。
2. **溯源可查询。** 对任何设计决定：哪个条款要求了它；对任何条款：哪个决策、哪次 prompt 会话或哪份外部标准孕育了它。
3. **Agent 基于基线工作。** Agent 任务引用 `spec-vX` + 条款 ID；给定同一基线的两个 agent，其产出设计的分歧小于给定环境聊天历史的两个 agent。
4. **演化是可读的。** 任意两个基线之间的 `ndf diff` 读起来像一份新团队成员能够吸收的、有意义的变更日志。
5. **不完整性的边界是可见的。** TBD、开放问题和覆盖缺口被计数、有趋势，而不是潜伏着。
6. **成本保持在理智范围内。** 规范工作量始终只占设计总工作量的适度比例——这正是中间道路的全部意义。如果维护规范的成本一旦逼近做设计本身，我们就重演了失败模式 B，此时必须削减严格度，而不是增加它。

附录 A——最小条款文法（非形式化）

```
clause        := heading id-anchor NEWLINE meta-comment NEWLINE body
id-anchor     := "{#" PREFIX "-" AREA "-" NUMBER "}"
meta-comment  := "<!-- ndf:" (key "=" value)+ "-->"
key           := "kind" | "level" | "layer" | "status" | "since"
               | "refines" | "depends-on" | "conflicts-with"
               | "verifies" | "origin" | "origin-status" | "model"
               | "affects" | "blocks" | "blocks-by" | "date" |
"author"
               | "couples-with" | "default" | "explore" | "unit"
body          := markdown，其中可包含：
                 [[ID]] | [[ID | text]]        交叉引用
                 ```lang ndf:normative ... ```  规范性代码岛
                 "⟨TBD: ...⟩"                  受追踪的空洞
                 "⟨DSE: ...⟩"                  设计空间参数（§4.8）
                 "> rationale: ..."            资料性的设计依据
```

工具链强制执行的文法规则：

- **每个条款恰好一条元数据注释。** 所有键都写进唯一的 `<!-- ndf: ... -->` 行；连续出现多条元数据注释是错误（`ndf check`：`multiple-meta`）。
- **`blocks-by`** 是问题条款 `blocks` 的反向边：条款可以声明阻塞它的开放 `Q-*` 项。
```

- **author** 在基础文法中可选；项目可以通过 `ndf.yaml` 中的 `lint: require-author: true` 要求 `decision`/`question` 条款必须带 author。
- **受追踪的空洞需要冒号形式。** 只有 `⟨TBD: ...⟩` / `⟨DSE: ...⟩` 计入 普查；裸的 `⟨TBD⟩` 记号只是对空洞的文字提及，不是空洞本身。
- **精化分层按传递闭包检查。** L2 条款可以经由一串 L2 `refines=` 边到达 L1。`def`/`arch`/`constraint` 条款（以及 `question`/`decision` 记录） 豁免于分层父子检查。Lint 发现分三个严重级别：**error**（违规）、 **warning**（依赖判断，`--strict` 时升级失败）、**note**——结构性 杂乱（例如从未溯源到 L0 意图的 L1 条款），仅供参考。

附录 B——精化序列的微型实例

- **L0** `{#FWD-000}` ——"交换机把帧转发到已知目的地所在的端口，目的地未知时泛洪，并学习源地址。" *（意图）*
- **L1** `{#FWD-LRN-001, refines=FWD-000}` ——"当在端口 P 上准入一个源地址为 S 的帧时，交换机 MUST 创建或刷新过滤数据库条目 (S → P)，老化时间遵循 [[CFG-AGE-001]]，除非 ⟨TBD: 静态条目覆盖策略⟩。" *（契约，带一个诚实的空洞）*
- **L2** `{#FWD-LRN-010, refines=FWD-LRN-001}` ——"过滤数据库 MUST 是一个 4 路组相联、16K 条目、以 {VLAN, MAC} 为键的哈希表；组溢出时 MUST 逐出刷新时间戳最老的条目。" *（机制）*
- **L3** `models/fdb.py` + `{#VER-LRN-101, verifies=FWD-LRN-001}` ——可执行模型加验收准则："Golden vector 套件 `lrn-basic` MUST 通过：10K 次随机的学习/老化/迁移事件，模型与 DUT 的过滤数据库在每一步都等价。" *（可执行）*

每一层都留在规范中；`ndf trace FWD-000` 打印这个序列；`ndf coverage` 确认 `FWD-LRN-001` 已被验证；那个 ⟨TBD⟩ 在 `Q-` 决议之前会一直出现在普查中。

附录 C——示例项目：两位 BCD 数字秒表

一份完整的、端到端的 NDF 规范，对象是一个刻意做小的设计，从而 §4–§5 中的每种机制都能完整展示而非零碎举例。设计对象：一个**数字秒表（digital chronometer）**，带一个**复位（reset）按钮、一个启动/停止（start/stop）按钮，以及以 BCD 输出驱动的两位秒数显示**。它虽小，却足以演练精化层、交叉引用、规范性代码、决策记录、开放问题与验证覆盖。

C.1 规范树

```
spec/
  ndf.yaml
  00-charter/
     charter.md            # CHR-000，范围、非目标
     glossary.md           # DEF-*
  20-behavior/
     counting.md           # CNT-*
     controls.md           # CTL-*
     display.md            # DSP-*
  30-interfaces/
     pins.md               # PIN-*
  40-constraints/
     timing.md             # CON-*
  50-verification/
     acceptance.md         # VER-*
  models/
     chrono.py             # 可执行参考模型
     test_chrono.py
  decisions/
     D-0001.md             # 在 99 回绕，而非 59
  open/
     Q-001.md              # 消抖间隔未定
```

ndf.yaml（节选）：

```
        project: chrono
        id-prefixes: [CHR, DEF, CNT, CTL, DSP, PIN, CON, VER, D, Q]
        layers: {L0: intent, L1: contract, L2: mechanism, L3:
executable}
```

C.2 章程 —— 00-charter/charter.md

```
        ## Chronometer intent {#CHR-000}
        <!-- ndf: kind=arch level=must layer=L0 status=stable
since=0.1 -->

        A digital chronometer measures elapsed time in whole seconds
while
        running, controlled by two momentary push-buttons — RESET and
        START/STOP — and presents the two-digit seconds count as BCD
outputs
        suitable for driving external 7-segment decoders.

        **Non-goals:** minutes/hours display, lap capture, sub-second
        resolution, power management.

        ## Definitions {#DEF-001}
        <!-- ndf: kind=def level=must layer=L1 status=stable since=0.1
```

```
-->

        - **running / stopped** — the two states of the chronometer;
the
          count advances only while *running*.
        - **BCD digit** — a 4-bit value in the range 0–9 encoding one
decimal
          digit, bit 3 = MSB.
        - **button press** — a debounced, single-clock-cycle assertion
event
          derived from a physical button ([[CTL-DEB-001]]).
```

C.3 接口 —— `30-interfaces/pins.md`

```
        ## External pins {#PIN-001}
        <!-- ndf: kind=req level=must layer=L1 status=stable since=0.1
-->

        The design MUST expose exactly the following interface:

        ```text ndf:normative
        clk        in   1   system clock, 32.768 kHz (see CON-CLK-001)
        rst_btn    in   1   RESET button, raw, active-high,
asynchronous
        ss_btn     in   1   START/STOP button, raw, active-high,
asynchronous
        sec_lo     out  4   BCD, seconds units digit (0-9)
        sec_hi     out  4   BCD, seconds tens digit  (0-9)
        running    out  1   1 while in running state (status
indicator)
```

Button inputs are raw mechanical-switch signals; conditioning is the design's responsibility ([[CTL-DEB-001]]).

C.4 行为 —— `20-behavior/`

`controls.md`：

```markdown
## Start/stop toggling {#CTL-SS-001}
<!-- ndf: kind=req level=must layer=L1 refines=CHR-000 status=stable
since=0.1 -->

Each press of START/STOP MUST toggle the state: stopped → running,
running → stopped. Stopping MUST preserve the current count; a
subsequent start MUST resume from the preserved count.

## Reset {#CTL-RST-001}
<!-- ndf: kind=req level=must layer=L1 refines=CHR-000 status=stable
since=0.1 -->
```
```

A press of RESET MUST set the count to 00 and MUST force the state to stopped, regardless of the current state. See [[D-0001 | D-0001]] for the rejected "reset keeps running" alternative.

Button conditioning {#CTL-DEB-001}
<!-- ndf: kind=req level=must layer=L1 refines=CHR-000 status=draft since=0.1 blocks-by=Q-001 -->

Each raw button input MUST be synchronized to `clk` (min. 2 flops) and debounced such that one physical press yields exactly one press event. A press event MUST be recognized no later than ⟨TBD: debounce interval, see Q-001⟩ after the physical press. Simultaneous RESET and START/STOP press events MUST resolve as RESET alone ([[CTL-RST-001]] wins).

counting.md：

```
        ## Counting contract {#CNT-001}
        <!-- ndf: kind=req level=must layer=L1 refines=CHR-000
status=stable since=0.2 -->

        While running, the count MUST increment by one exactly once
per
        elapsed second, with long-term rate accuracy limited only by
the
        clock source ([[CON-CLK-001]]). The count MUST hold at its
value
        while stopped. On incrementing past 99 the count MUST wrap to
00 and
        continue ([[D-0001]]).

        ## Second-tick generation {#CNT-TCK-010}
        <!-- ndf: kind=req level=must layer=L2 refines=CNT-001
status=stable since=0.2 -->

        A modulo-32768 divider on `clk` MUST generate a one-cycle
`tick`
        pulse each second. RESET ([[CTL-RST-001]]) MUST also clear the
        divider, so the first second after reset is full-length.

        ## BCD counter {#CNT-BCD-010}
        <!-- ndf: kind=req level=must layer=L2 refines=CNT-001
status=stable since=0.2 -->

        The count MUST be maintained as two cascaded decade counters
(units,
        tens), never holding a non-BCD value; on `tick` while running:
        units 9→0 carries into tens, tens 9→0 wraps the whole count to
00.
```

display.md：

```
## Display encoding {#DSP-001}
<!-- ndf: kind=req level=must layer=L1 refines=CHR-000
status=stable since=0.1 -->

`sec_hi`/`sec_lo` MUST continuously present the current count
as BCD
([DEF-001]) with no blanking, multiplexing, or intermediate
non-BCD codes observable at the outputs; the pair MUST update
atomically within one `clk` cycle.
```

C.5 约束 —— 40-constraints/timing.md

```
## Clock source {#CON-CLK-001}
<!-- ndf: kind=constraint level=must layer=L1 status=stable
since=0.1 -->

The system clock is 32.768 kHz (watch crystal). The design
MUST meet
timing at this frequency; it MAY be functional above it.
```

C.6 可执行模型 —— models/chrono.py（由 CNT-001 链接）

```
# Authoritative for control/count semantics at one-tick
granularity.
# Debounce and clock division are below this model's
abstraction.
class Chrono:
    def __init__(self):
        self.count, self.running = 0, False

    def press_reset(self):            # CTL-RST-001
        self.count, self.running = 0, False

    def press_startstop(self):        # CTL-SS-001
        self.running = not self.running

    def tick(self):                   # CNT-001, one call per
second
        if self.running:
            self.count = (self.count + 1) % 100  # D-0001:
wrap at 99

    @property
    def bcd(self):                    # DSP-001
        return (self.count // 10, self.count % 10)
```

C.7 验证 —— 50-verification/acceptance.md

```
## Control sequence equivalence {#VER-CTL-001}
<!-- ndf: kind=verif level=must layer=L3 verifies=CTL-SS-001,CTL-RST-001,CNT-001 -->

The DUT MUST match `models/chrono.py` on 1,000 randomized sequences
of {press_reset, press_startstop, tick} (10,000 events each),
comparing `(count, running)` after every event.

## Wrap behavior {#VER-CNT-002}
<!-- ndf: kind=verif level=must layer=L3 verifies=CNT-001,CNT-BCD-010 -->

Directed test: from reset, run 100 ticks; outputs MUST read 99 at
tick 99 and 00 at tick 100 with `running` still asserted.

## Output BCD invariant {#VER-DSP-003}
<!-- ndf: kind=verif level=must layer=L3 verifies=DSP-001 -->

Assertion, all tests: `sec_hi <= 9 && sec_lo <= 9` at every clock
edge, including during carry propagation.
```

注意这个有意留下的缺口：`CTL-DEB-001` 目前还没有 `verifies` 链接——`ndf coverage` 会报告它，在 `Q-001` 解决并补上消抖测试之前，它一直亮红。不完整性的边界是可见的，正如 §4.7 所述。

C.8 决策记录与开放问题

`decisions/D-0001.md`：

```
# D-0001: Count wraps at 99; reset forces stop {#D-0001}
<!-- ndf: kind=decision date=2026-07-22 affects=CNT-001,CTL-RST-001 -->

**Context.** Two-digit display; the seconds field could wrap at 59
(clock-like) or 99 (full range). Also debated: should RESET while
running restart the count without stopping ("flying reset")?

**Decision.** Wrap at 99 — with no minutes digit, a 59-wrap conveys
no extra information and wastes 40% of display range. RESET forces
stopped — matches user expectation of a "clear" operation.
```

```
        **Alternatives rejected.** Wrap at 59 (clock semantics without
a
        clock); flying reset (surprising, and complicates VER-CTL-
001).

        **Source.** Human–agent session 2026-07-22, cycle 2;
supersedes the
        cycle-1 prompt "make it count like a clock."
```

open/Q-001.md：

```
        # Q-001: Debounce interval {#Q-001}
        <!-- ndf: kind=question status=open blocks=CTL-DEB-001
date=2026-07-22 -->

        Datasheet for the chosen buttons not yet available; bounce
time
        unknown. Candidate: 10 ms (327 cycles @ 32.768 kHz).
Resolution will
        set the ⟨TBD⟩ in CTL-DEB-001 and add a debounce acceptance
test.
```

C.9 工具怎么说

在这个基线（spec-v0.2）上，工具链的输出诚实地总结了现状（以下为参考 实现在 examples/chrono/ 上输出的逐字摘录；coverage 块节选了与此处相关的行）：

```
$ ndf trace CHR-000
CHR-000 (L0)
├── CTL-SS-001 (L1) ── verified by VER-CTL-001
├── CTL-RST-001 (L1) ── verified by VER-CTL-001
├── CTL-DEB-001 (L1, draft, 1 TBD, blocked by Q-001)   ⚠ unverified
├── CNT-001 (L1) ── verified by VER-CNT-002, VER-CTL-001
│   ├── CNT-TCK-010 (L2)
│   └── CNT-BCD-010 (L2) ── verified by VER-CNT-002
└── DSP-001 (L1) ── verified by VER-DSP-003

$ ndf coverage
L1 must contracts:    8
  verifies-linked:    4 (4 unverified)
  unverified:         CON-CLK-001, CTL-DEB-001, DEF-001, PIN-001
TBD markers:          1
conflicts:            0
open questions:       1 of 1
```

"L1 must contracts" 统计所有 layer=L1 level=must 的条款，不区分 kind——定义和约束同样欠着验证链接，或者欠一条"无需验证"的明确决策；无论哪种情况，普查都让它们保持可见。CTL-DEB-001 是唯一的*行为性*缺口 （§C.7 有意留下的空

洞）；`ndf check` 还会把 PIN-001 报告为结构性杂乱 （一个从未溯源到 L0 意图的 L1 根条款）——这是一条 note，不是违规。

即使在这个玩具尺度上，收益模式已经清晰可见：第 1 周期那句"让它像时钟一样计数"的 prompt 没有消失在聊天记录里——它被一条有记录的决策（`D-0001`）取代，且有两个条款引用了这条决策；那个唯一真正悬而未决的工程参数是一个受追踪的空洞，而不是一次伏击；而一个被要求"实现计数器"的 agent，可以精确地拿到 `CNT-*` 及其一跳邻域（`CTL-RST-001`、`CON-CLK-001`、`D-0001`），而不是一份聊天转录。